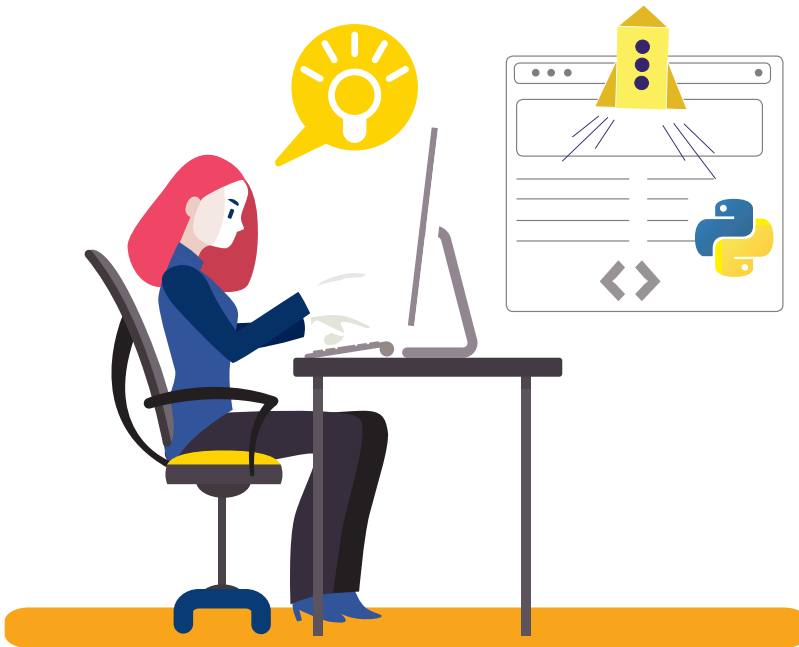


Deploying a Flask Application via the Apache Server

This tutorial will guide you on how to deploy a Python application using the Apache server. It also demonstrates how to use WSGI to allow Apache to interact with Python.



Python

Python is a great, easy-to-learn, and all-powerful programming language, and with frameworks such as Django and Flask, we can use it to create full-fledged web applications. Once we create a web application using libraries and frameworks such as Flask, we need to host it on a server and expose it to the public.

Flask

Flask is a micro web framework for Python. It is provided with its own http server, and is not a native language. It has limited capacity. To prevent overloading, we must use Apache, which will use WSGI files to access Flask applications. This will let Apache interact with Python as if it is a native.

Apache

Apache is a high-performance, free and open source HTTP server software and is ranked to be the second most popular after Nginx. However, Apache doesn't understand Python script. This is where WSGI is needed.

There are various applications that can work as a local server. Free and open source servers such as XAMPP, WAMP, LAMP and MAMP are a group of tools that run together to make a complete server environment. The 'P' in each stands for PHP and 'A' for Apache. However, such resources are limited for Python, so developers can take advantage of using Apache to connect to Python in different operating systems. Python's frameworks are inbuilt with their own server, but this server is just for testing and is not fit for a production environment. Once in production, we need a real server like Apache. This server takes care of the load balancing and scaling for the deployed application.

This tutorial focuses on deploying a Python application along with Apache in a Linux (Ubuntu) based environment. I would suggest you try this out on a development environment and not a production server.

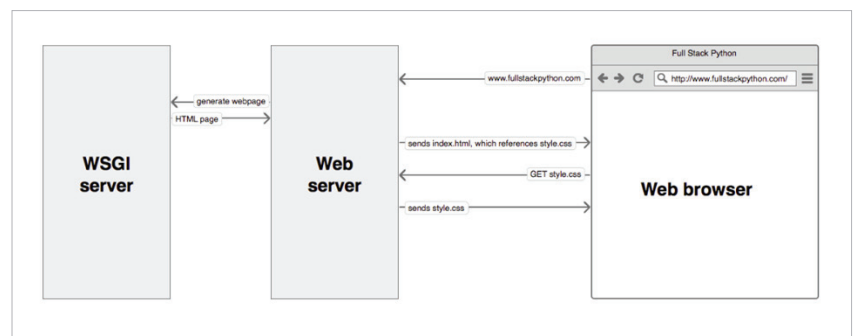


Figure 1: Request handling